

Компьютерная дисморфология: распознавание генетических синдромов по лицу

Отчёт о продукте и технической реализации (MVP)

Платформа: `frontend.rusted-apple.com` · API: `api.rusted-apple.com`

Дата: 30 мая 2026 г. · Статус: MVP, развёрнут в продакшн-окружении

О документе. Это сводный отчёт о продукте Phenograph (внутреннее кодовое имя бэкенда — *Oblivion*). Описание архитектуры опирается на проектные материалы из каталога `design/` (целевое видение системы), а описание реализации — на фактический исходный код репозитория и на наблюдения, сделанные на живом стенде. Все экранные снимки получены с работающего развёрнутого приложения. Документ намеренно честно разграничивает *реализованное*, *частично реализованное* и *целевое*.

Содержание

1. Краткое резюме
2. Контекст и рыночная возможность
3. Продукт: возможности и пользовательские сценарии
4. Архитектура системы
5. Бэкенд: микросервисы и данные
6. Ядро распознавания (ML): текущее состояние и цель
7. Фронтенд: Phenograph SPA
8. Безопасность
9. Развёртывание MVP
10. Текущие ограничения и известные проблемы
11. Дорожная карта
12. Приложения (полная карта API, каталог синдромов, доступы)

1. Краткое резюме

Phenograph — это платформа поддержки клинических решений для медицинской генетики. По одной фотографии лица пациента система выделяет морфологические признаки и формирует ранжированный список наиболее вероятных генетических синдромов, связанных с онтологией человеческого фенотипа (Human Phenotype Ontology, HPO). По сути это русско-/локально-ориентированный функциональный аналог мирового лидера рынка — Face2Gene (компания FDNA).

Продукт обслуживает **две роли пользователей** по проверенной бизнес-модели Face2Gene:

- **Врач-генетик** — создаёт клинические кейсы, загружает изображение, получает дифференциальный диагноз (ранжированный список синдромов с оценками), работает со справочником синдромов и HPO.
- **Лаборатория** — пакетная (batch) обработка изображений, подписки и тарифы, учёт потребления.

Технически система построена как набор из **7 независимых микросервисов на Rust (фреймворк Axum)** с асинхронной обработкой через **Apache Kafka** и одностраничным фронтендом (SPA) на **React**. Сервисы спрятаны за единым API-шлюзом (nginx). Хранилища: PostgreSQL, Redis, объектное хранилище MinIO (S3), Elasticsearch.

Ключевая оговорка. На текущем этапе MVP **само распознавание — это заглушка (stub)**: сервис `recognition` игнорирует содержимое изображения и возвращает случайный top-10 синдромов из каталога на 50 позиций после имитации задержки «инференса» в 0,5–2 с. При этом *контракты сообщений и весь конвейер обработки — промышленного качества*, поэтому замена заглушки на реальную ML-модель не требует переделки остальной системы.

Ключевые цифры реализации

Параметр	Значение
Микросервисов (Rust/Axum)	7 + общая библиотека <code>oblivion-common</code>
Схем в PostgreSQL	6 (auth, cases, labs, billing, notifier, reference)
HTTP-операций API	24 бизнес-операции + health-проверки
Топиков Kafka	3 рабочих (<code>analysis.requests</code> , <code>analysis.results</code> ,

notifications)

Синдромов в каталоге
распознавания

50 (со стабильными UUID)

Справочные данные

50 синдромов + 100 НРО-терминов (сид)

Статус развёртывания

Живой стенд (frontend + API + TLS), демо-доступы засеяны

2. Контекст и рыночная возможность

Рынок распознавания генетических заболеваний по изображению лица **консолидирован вокруг единственного лидера — FDNA с продуктом Face2Gene**. Им пользуются около 70% генетиков мира в более чем 10 000 медицинских центрах 130 стран. Алгоритм DeepGestalt показывает 91% точности попадания в первую десятку диагнозов (Nature Medicine, 2019); база содержит свыше 150 000 изображений и более 5 000 заболеваний.

Бизнес-модель Face2Gene — фриимиум с сетевым эффектом: приложение CLINIC бесплатно для врачей (в обмен на данные для обучения), монетизация — через продукт LABS для лабораторий по тарифной сетке:

Тариф	Кейсов в месяц	Превышени е
Bronze	40	\$50/кейс
Silver	80	\$50/кейс
Gold	145	\$50/кейс
Platinum	300	\$50/кейс
Bundle	40–300	\$40/кейс

Именно эта сетка тарифов воспроизведена в биллинг-сервисе Phenograph (см. раздел 5).

Рыночная возможность для Phenograph. В России и СНГ собственных решений в области фенотипирования по лицу **не существует**. Face2Gene используется отдельными генетиками в индивидуальном порядке, а **главным барьером для распространения называют отсутствие русскоязычного интерфейса**. Сегмент ИИ-диагностики редких заболеваний растёт темпами ~29% в год и достигнет ≈\$5,5 млрд к 2029 г. Phenograph позиционируется как локализованный аналог с той же двусторонней бизнес-моделью (бесплатно врачам, платно лабораториям) и потенциалом первопроходца на недоооохваченном рынке.

Полный конкурентный и рыночный анализ приведён в исходном материале [design/compass_artifact...md](#) и не дублируется здесь целиком.

3. Продукт: возможности и пользовательские сценарии

Ниже — обзор реального интерфейса, снятый с работающего стенда `frontend.rusted-apple.com`. Приложение распознаёт роль пользователя (врач / лаборатория) и показывает соответствующую навигацию.

3.1. Маркетинговый лендинг

Публичная посадочная страница выполнена как «кинематографичный» прокручиваемый рассказ из трёх актов (двойная спираль ДНК → лицевая сетка из ориентиров → панель доказательной базы), за которым следуют блок возможностей и блок доказательств. Визуальный язык — сдержанный, «клинический», с акцентом на доверие (бейджи HIPAA / SOC 2 / GDPR, формулировка «Not a diagnostic device. Clinical decision support only»).

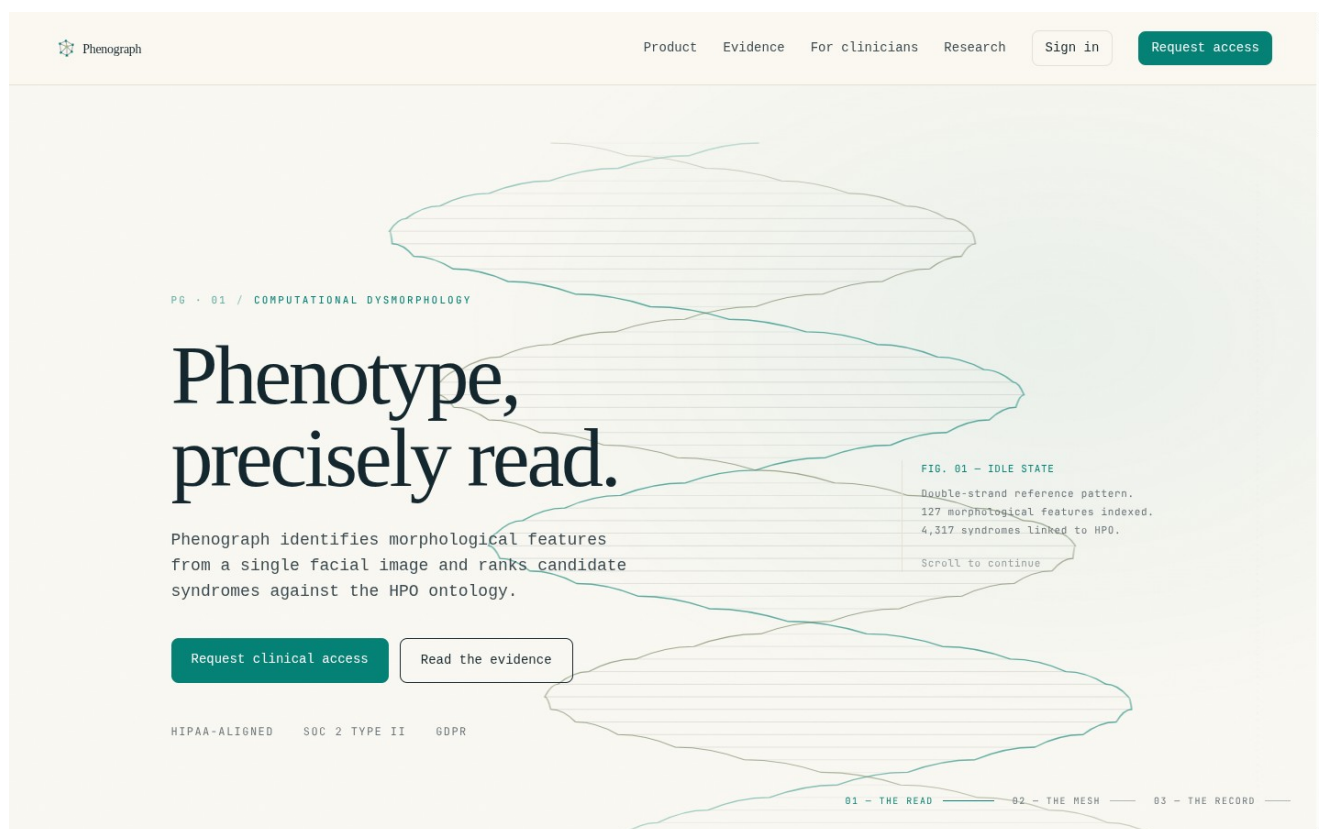


Рис. 1 — Главный экран лендинга: «Phenotype, precisely read.» с анимированной спиралью ДНК.

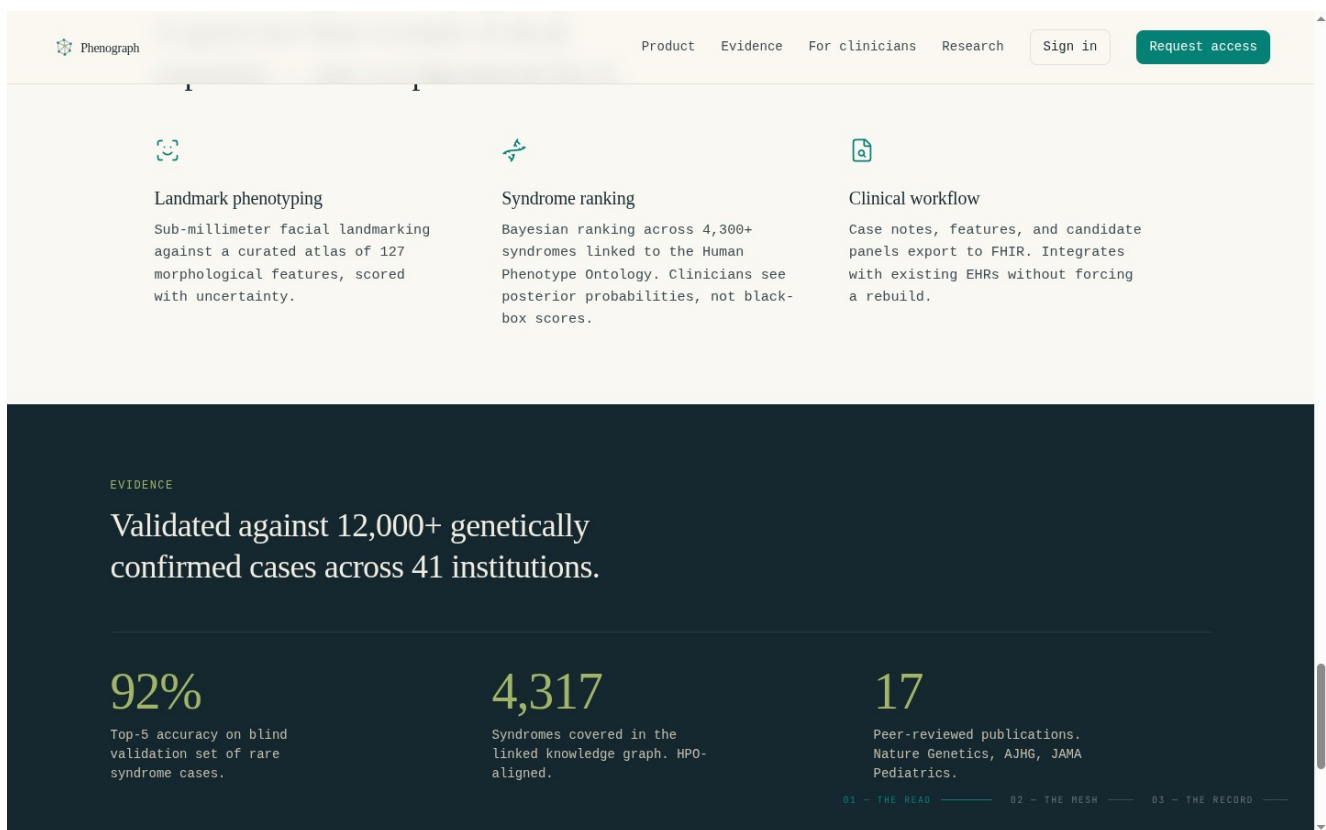


Рис. 2 — Блок возможностей: фенотипирование по ориентирам, ранжирование синдромов, клинический workflow.

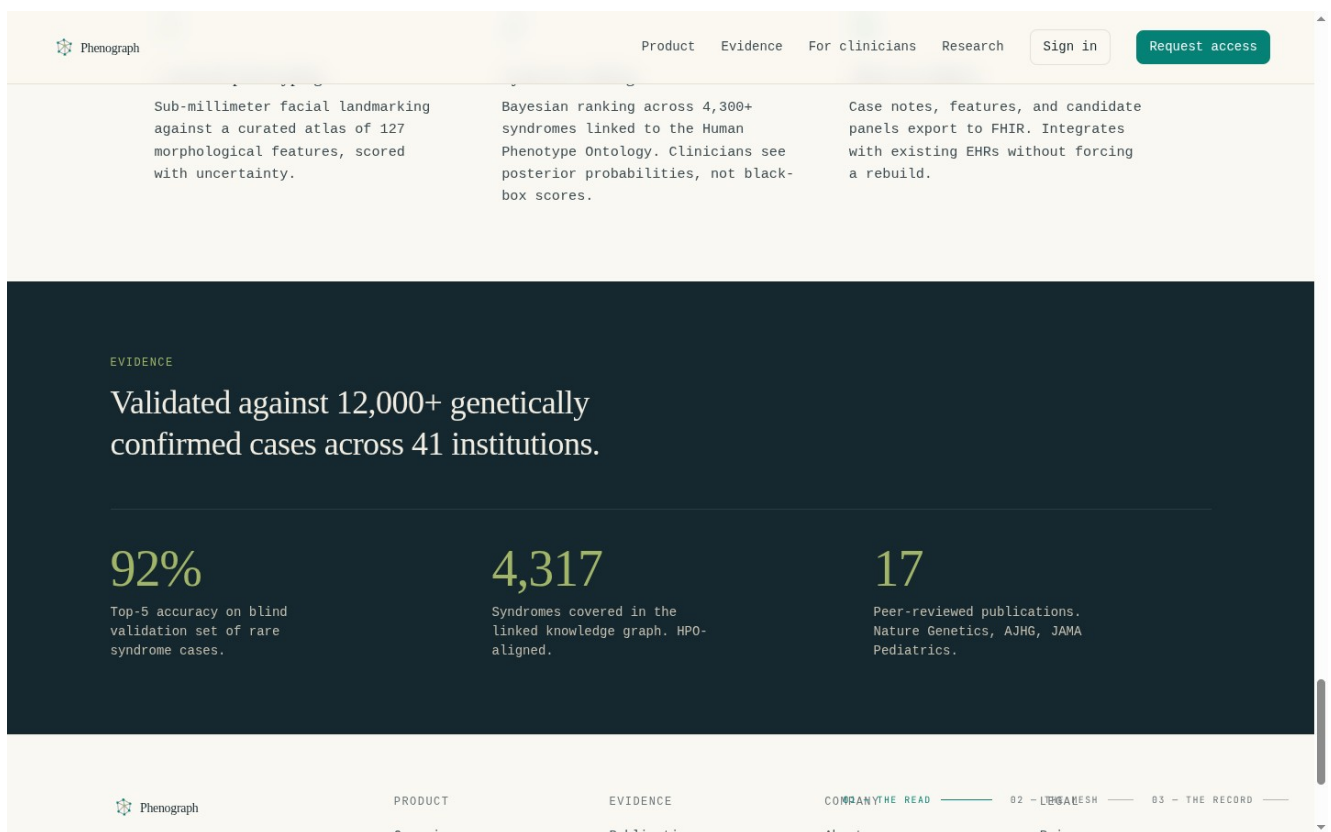


Рис. 3 — Блок доказательной базы.

Важно для отчёта. Цифры на лендинге (точность 92% top-5, 4 317 синдромов, 12 000+ подтверждённых случаев, 17 публикаций, 41 учреждение) — это **маркетинговые заявления, отражающие целевое видение продукта, а не текущую реализацию.** Фактический MVP-бэкенд оперирует каталогом на 50 синдромов и использует заглушку распознавания (см. раздел 6).

3.2. Аутентификация

Вход и регистрация по e-mail и паролю. Доступ к рабочему пространству ограничен (защищённые маршруты). После входа access-токен (JWT) хранится в браузере и автоматически обновляется по refresh-токену при истечении.

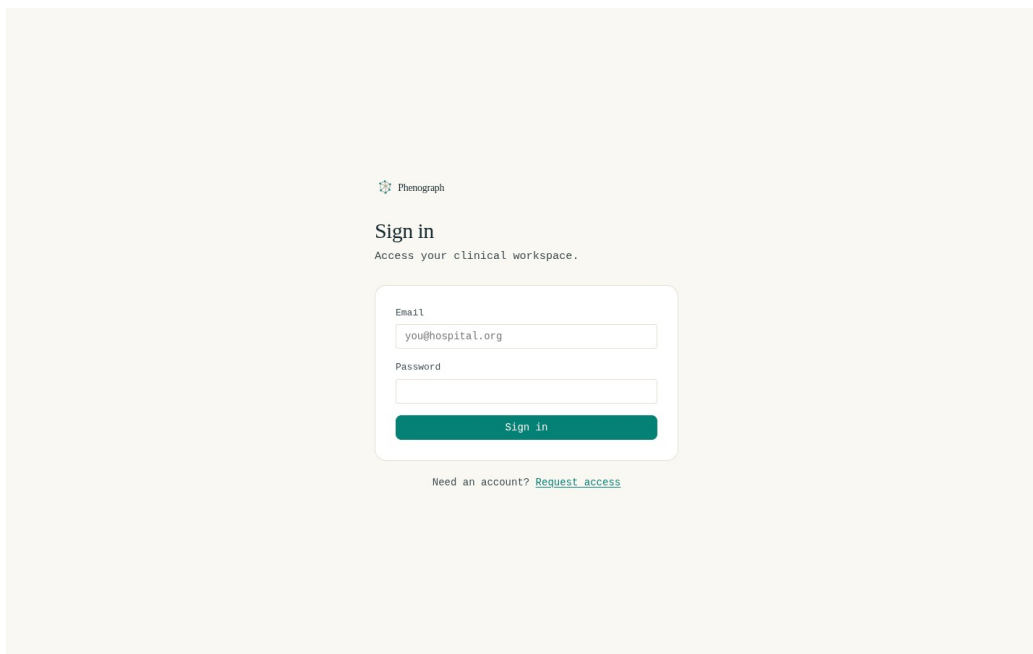


Рис. 4 — Экран

входа в клиническое рабочее пространство.

3.3. Сценарий врача

Домашний экран врача — список кейсов с фильтрами по статусу (все / в обработке / завершён / ошибка) и таблицей: пациент, возраст, статус, дата.

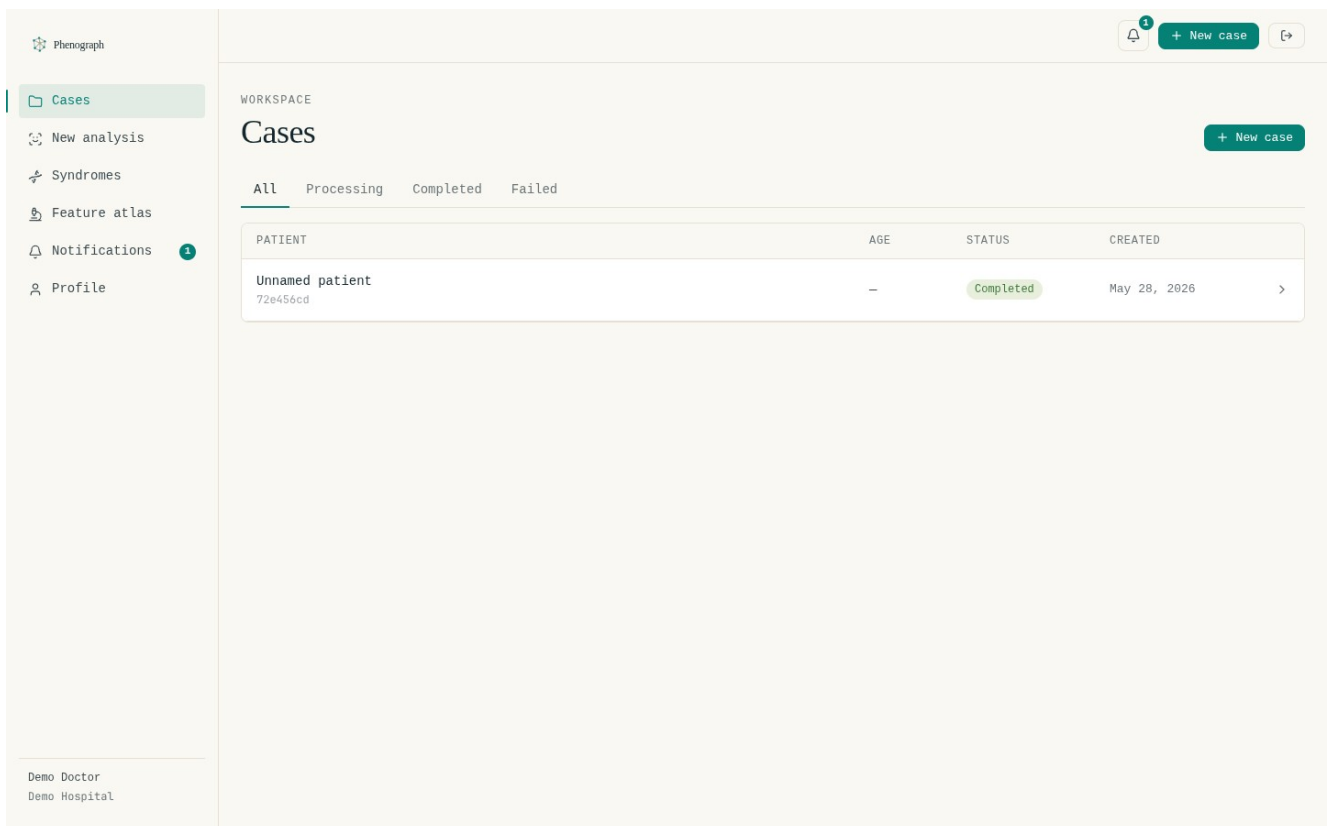


Рис. 5 — Рабочее пространство врача: список клинических кейсов.

Создание нового анализа: метаданные пациента → загрузка изображения по защищённой (presigned) ссылке напрямую в объектное хранилище → подтверждение загрузки запускает асинхронный анализ.

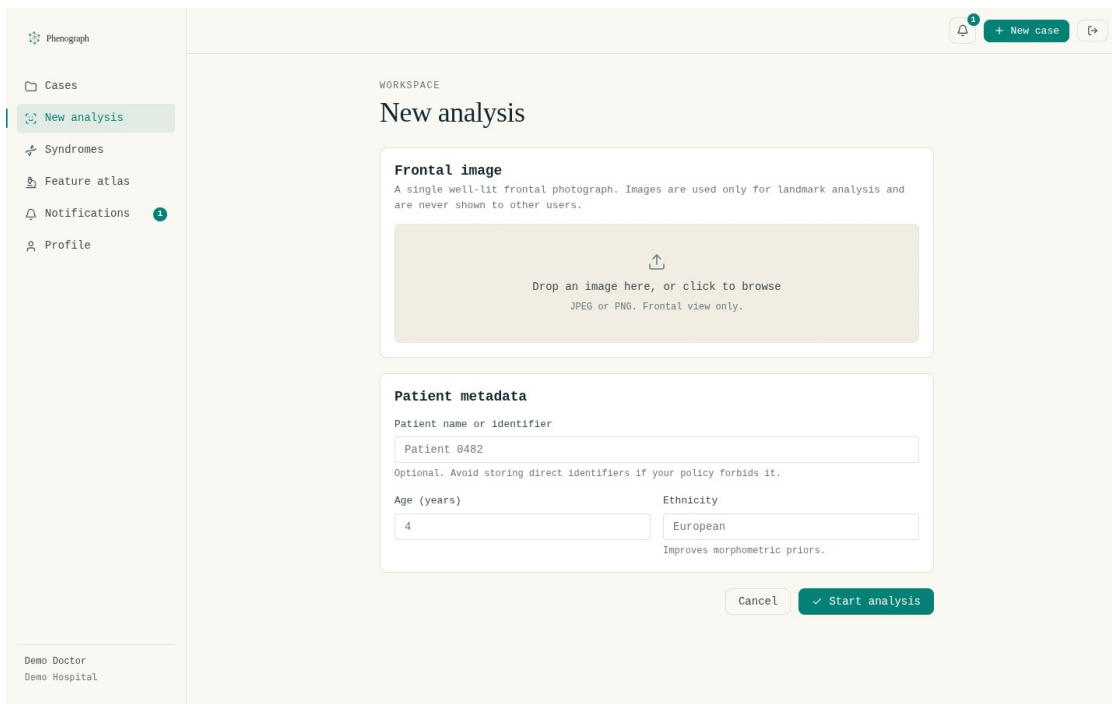


Рис. 6 —

Мастер создания кейса и загрузки изображения («New analysis»).

Ключевой экран продукта — карточка кейса с результатом. Слева — абстрактная визуализация лицевой сетки из ориентиров (фотография пациента принципиально не отображается — «For privacy, the patient photograph is never rendered»). Справа — ранжированный список из 10 синдромов-кандидатов с апостериорными оценками и индикаторами уверенности.

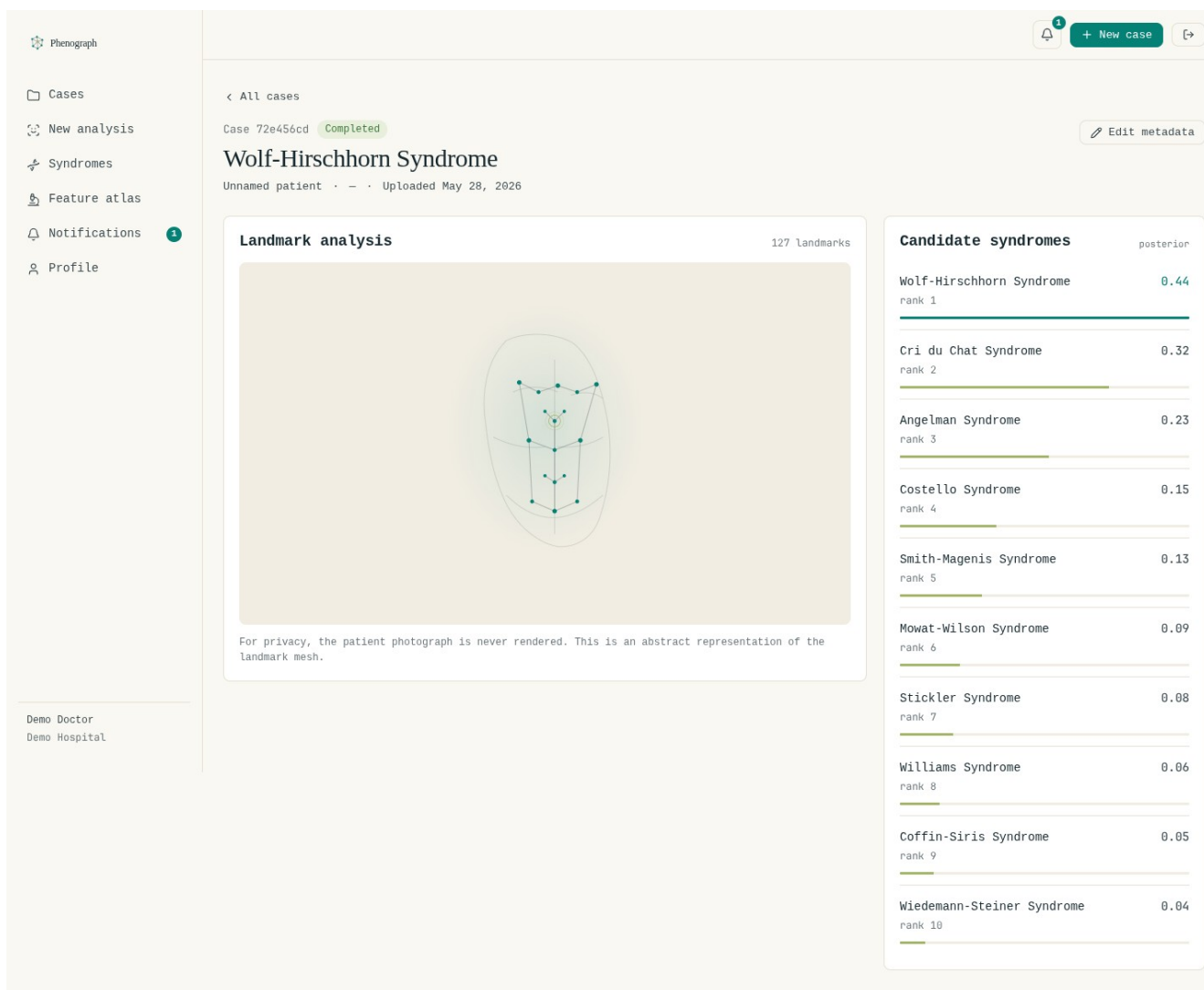


Рис. 7 — Карточка завершённого кейса: лицевая сетка + дифференциальный диагноз (top-10).

3.4. Справочник синдромов и атлас признаков (HPO)

Раздел доступен обеим ролям. Справочник синдромов — поиск и постраничный просмотр; карточка синдрома содержит описание, OMIM-идентификатор, распространённость, тип наследования и связанные HPO-термины. Отдельный «атлас признаков» — поиск по терминам Human Phenotype Ontology.

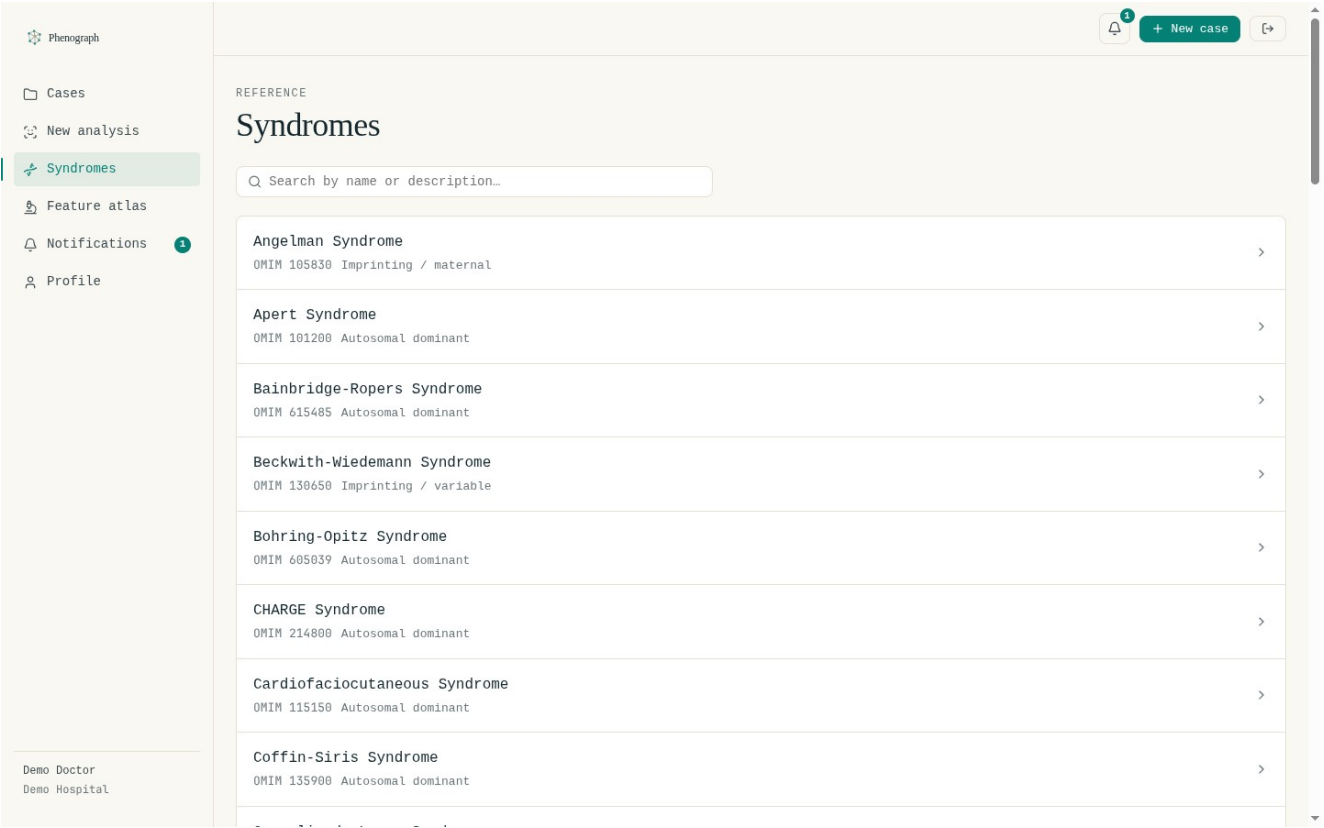


Рис. 8 — Справочник синдромов с поиском.

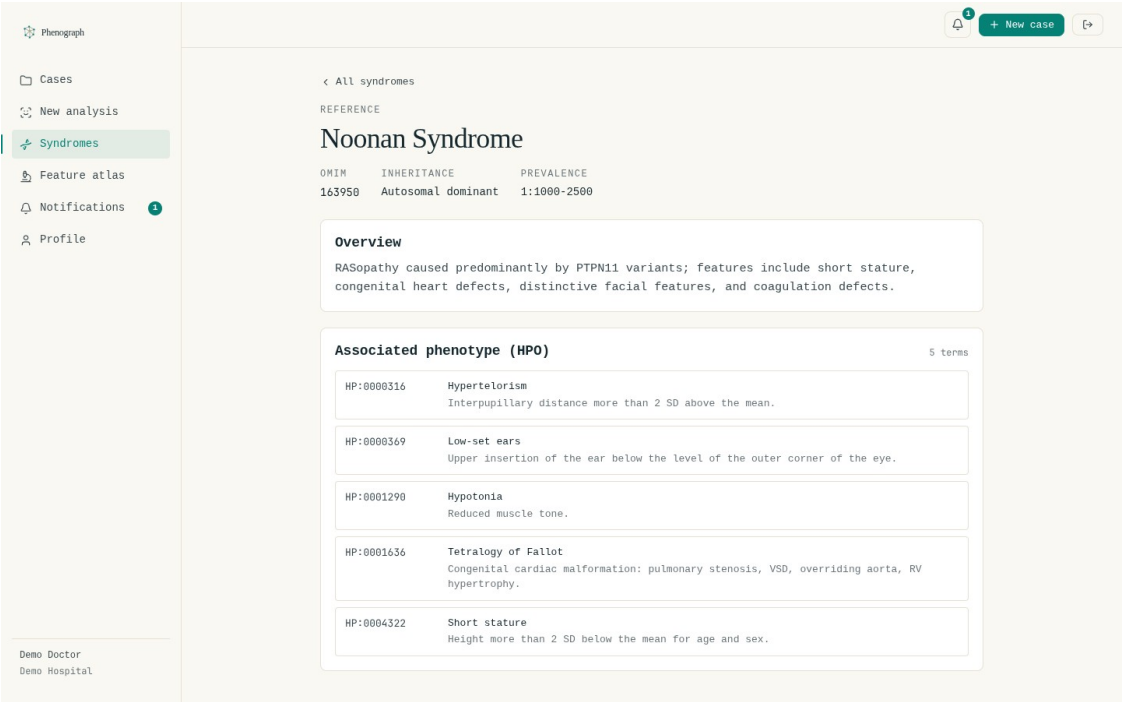


Рис. 9 — Карточка синдрома с описанием и связанными HPO-признаками.

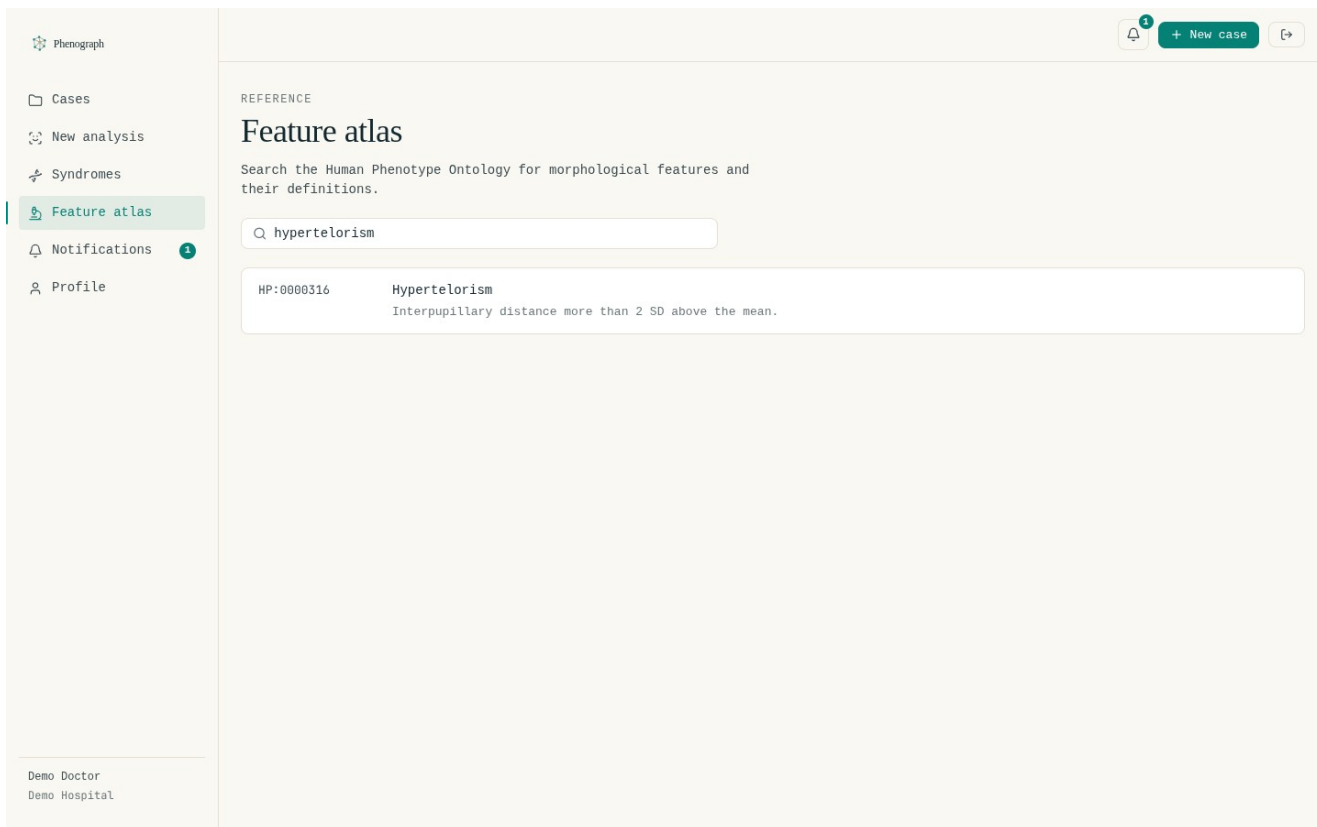


Рис. 10 — Атлас признаков (поиск по НРО), пример запроса «hypertelorism».

3.5. Сценарий лаборатории

Лаборатория работает с пакетами (batch) изображений: постановка изображений в очередь, отправка батча, просмотр результатов по мере обработки, помесечный учёт потребления и выбор тарифа. Навигация лаборатории отличается от врачебной (Batches, New batch, Usage, Billing).

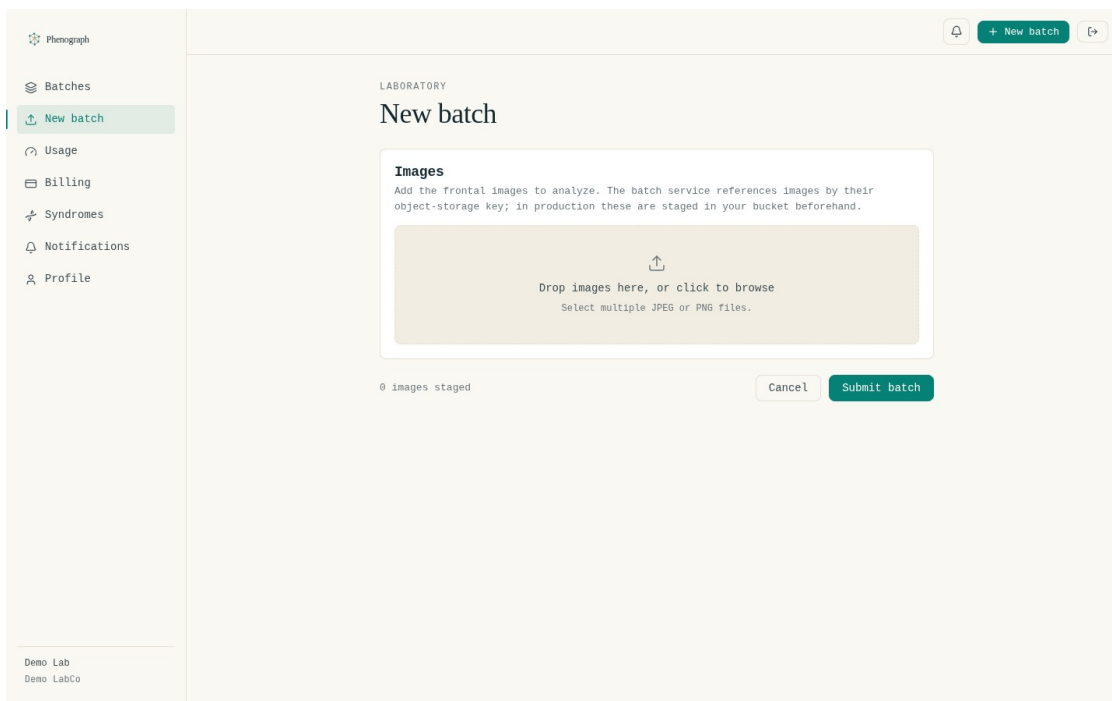


Рис. 11 —
Лаборатория: создание нового пакета (batch) изображений.

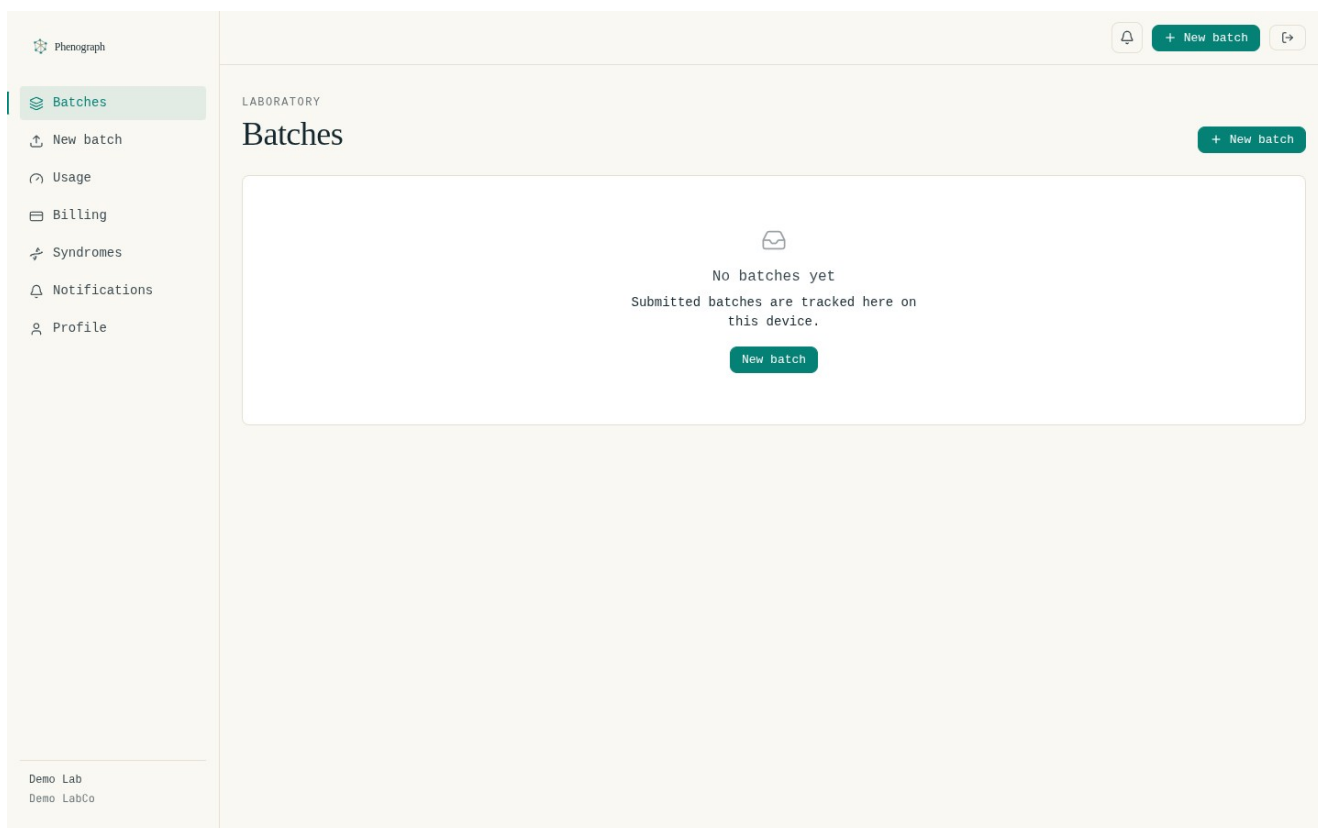


Рис. 12 — Лаборатория: список пакетов.

Замечание: на живом стенде у демо-лаборатории отсутствует активная подписка, а биллинг-страница в текущей сборке падает; в результате лабораторный/монетизационный сценарий не воспроизводится полностью end-to-end. Подробности — в разделе 10.

3.6. Уведомления и профиль

Уведомления формируются автоматически (например, «анализ готов») и отображаются в приложении с возможностью отметить прочитанным. Профиль показывает данные учётной записи.

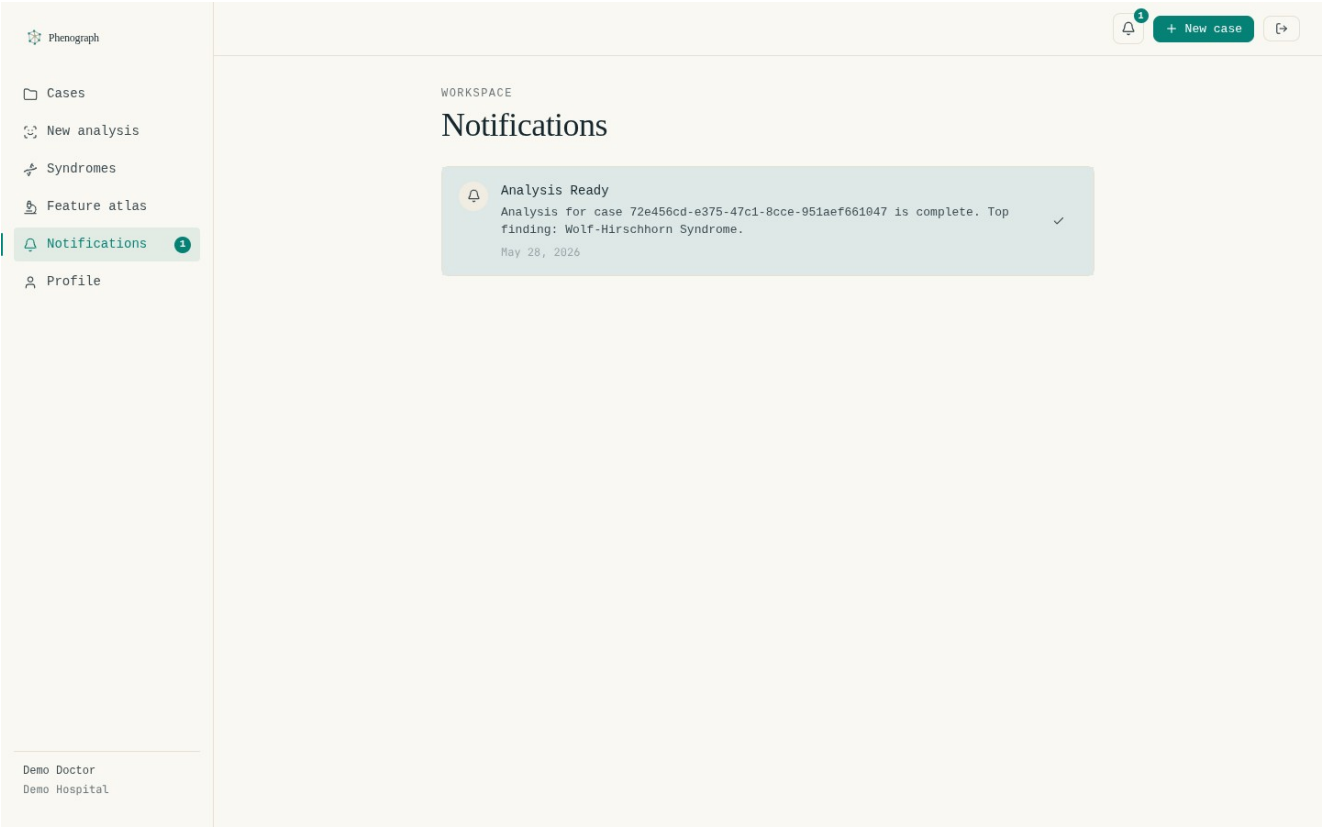


Рис. 13 — Центр уведомлений.

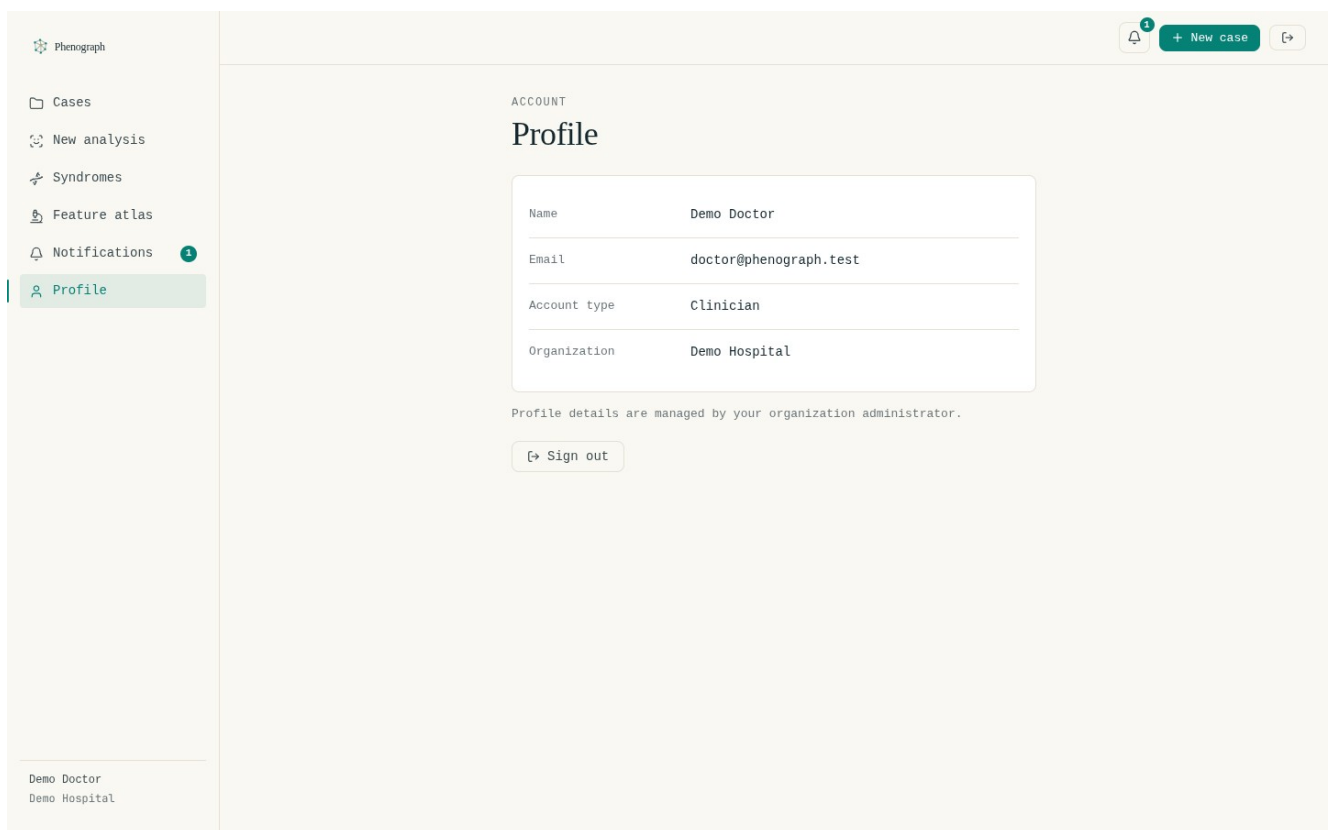


Рис. 14 — Профиль пользователя (только просмотр — см. раздел 10).

4. Архитектура системы

4.1. Целевая архитектура (видение)

Проектные материалы (**design/**) описывают целевую облачную архитектуру: клиенты (web на React, мобильное приложение на Flutter, Labs API по REST/gRPC) → пограничный слой (CDN, балансировщик) → кластер Kubernetes с API-шлюзом (Kong/Traefik) и ограничителем запросов → микросервисы (Core: Auth, Case, Labs; Async: Recognition на GPU, Notifier; Support: Reference, Billing) → шина Apache Kafka → хранилища (PostgreSQL, S3/MinIO, Redis, Elasticsearch, векторная БД эмбедингов лица, реестр моделей MLflow) → наблюдаемость (Prometheus, Grafana, Jaeger, ELK).

Диаграмма целевой архитектуры приведена в проектных материалах **design/** и здесь не дублируется.

Ключевые потоки данных в целевой модели:

- **Создание кейса врачом:** клиент → шлюз → Case Service → PostgreSQL + S3 → Kafka (**analysis.requests**).
- **Анализ изображения:** Kafka → Recognition Service → векторная БД → Kafka (**analysis.results**) → Case Service.
- **Пакет лаборатории:** Labs API → шлюз → Labs Service → Kafka → Recognition → результат.
- **Уведомления:** Kafka (**notifications**) → Notifier → Email/Push/Webhook.

4.2. Реализованная архитектура и развёртывание в Kubernetes

Платформа реализована как 7 независимых сервисов на Rust/Axum за единым API-шлюзом (nginx), с инфраструктурой PostgreSQL / Apache Kafka / Redis / MinIO / Elasticsearch. Для продакшна **подготовлены и проверены манифесты Kubernetes**: сервисы разворачиваются как Deployments за Ingress (TLS), а stateful-компоненты (БД, шина, хранилища) — как StatefulSets/Deployments внутри кластера. Это приближает реализацию к целевой архитектуре из §4.1.

Статус развёртывания. Манифесты Kubernetes готовы к продакшн-развёртыванию. Публичный демо-стенд `rusted-apple.com`, с которого сделаны экранные снимки, на момент подготовки отчёта работает на упрощённом одномашинном варианте (systemd-юниты + Docker-

инфраструктура, скрипт `deploy.sh`) — см. §9.

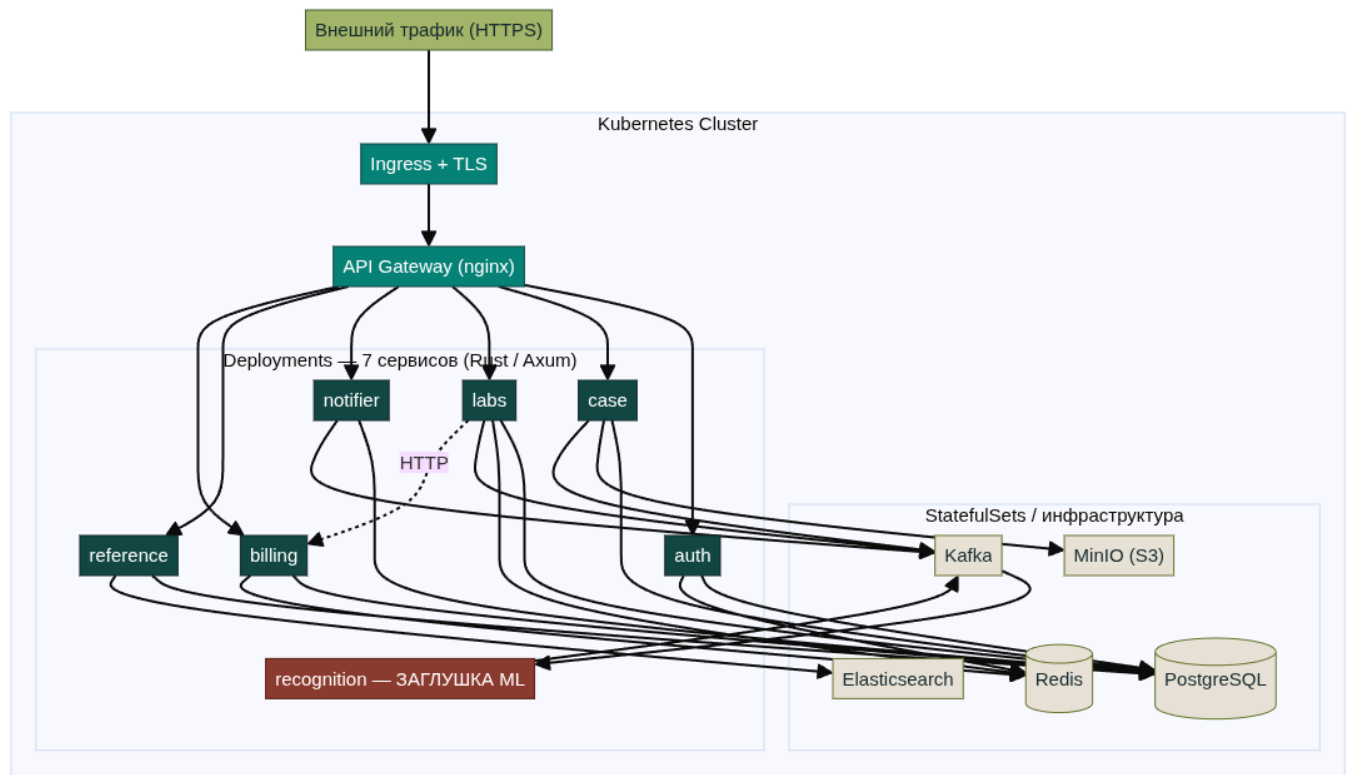


Рис. 15 — Развёртывание в Kubernetes (манифесты подготовлены). Красным выделена заглушка распознавания.

Слой	Реализация
Клиент	React SPA (Phenograph), <code>frontend.rusted-apple.com</code>
Вход в кластер	Ingress + TLS (демо-стенд: системный nginx + Let's Encrypt); CORS, rate-limit, заголовки безопасности (CSP/HSTS)
API-шлюз	nginx, маршрутизация по префиксу пути
Сервисы	7 сервисов Rust/Axum (порты 3001–3007) как Deployments (демо-стенд: <code>systemd</code> , <code>Restart=always</code>)
Шина сообщений	Apache Kafka 3.7
Хранилища	PostgreSQL 16, Redis 7, MinIO (S3), Elasticsearch 8 — как StatefulSets/тома в кластере

4.3. Сквозной поток анализа (end-to-end)

Поток врача

1. `POST /cases` — создаётся кейс (статус `pending`), возвращается `presigned-URL` для загрузки.
2. Браузер кладёт изображение `PUT`-запросом прямо в MinIO/S3 (без участия сервера).
3. `POST /cases/{id}/confirm-upload` — сервис проверяет наличие объекта (`HEAD`), переводит кейс в `processing` и публикует `AnalysisRequest` в топик `analysis.requests`.
4. Recognition-сервис читает запрос из Kafka, имитирует инференс, публикует `AnalysisResult` в `analysis.results`.
5. Case-сервис потребляет результат: сохраняет ранжированные диагнозы, переводит кейс в `completed`, публикует уведомление в `notifications`.
6. Notifier сохраняет уведомление в БД (и, при настроенном SMTP, отправляет письмо).
7. Фронтенд опрашивает `GET /cases/{id}` каждые 2 с, пока статус не станет `completed`, и показывает диагнозы.

Поток лаборатории

1. `POST /labs/batch` — проверка роли `lab` → ограничитель запросов (Redis) → проверка квоты в биллинге → создание пакета и его элементов → по одному `AnalysisRequest` на каждое изображение (`source = lab`).
2. Recognition обрабатывает каждый запрос и возвращает результат с тем же `batch_id`.
3. Labs-сервис потребляет результаты (фильтруя `source = lab`), помечает элементы пакета как `completed` и инкрементирует счётчик обработанных; когда обработаны все — пакет становится `completed`.

5. Бэкенд: микросервисы и данные

5.1. Общие принципы

Все сервисы написаны на Rust поверх Axum и разделяют общую библиотеку `oblivion-common`, которая задаёт единые контракты:

- **Единый JSON-конверт ответа** `ApiResponse`: `{ "status": "ok"|"error", "data": ...|null, "error": ...|null }`.
- **Единый тип ошибки** `ServiceError` с отображением на HTTP-коды (404/400/401/403/409/500).
- **Строго типизированные идентификаторы** (`UserId`, `CaseId`, `LabId`, `SyndromeId`, `BatchId`).
- **Доменные перечисления** `Role` (`doctor/lab/admin`), `AnalysisStatus`, `AnalysisSource`.
- **Константы топиков и структуры сообщений Kafka** (`AnalysisRequest`, `AnalysisResult`, `DiagnosisEntry`, `NotificationMessage`).
- **Слой CORS**, управляемый переменной `CORS_ALLOWED_ORIGIN`.

Качество кода: запрещён `unsafe`, включён строгий линт (`clippy::pedantic`), модули покрыты юнит-тестами (сериализация контрактов, генератор диагнозов, биллинговая арифметика и т.д.).

Единый конверт ответа (`crates/oblivion-common`):

```
// один и тот же конверт возвращает каждый HTTP-эндпоинт платформы
pub struct ApiResponse<T: Serialize> {
    pub status: &'static str,    // "ok" | "error"
    pub data: Option<T>,
    pub error: Option<String>,
}

impl<T: Serialize> ApiResponse<T> {
    pub fn ok(data: T) -> Self {
        Self { status: "ok", data: Some(data), error: None }
    }

    pub fn error(msg: impl Into<String>) -> Self {
        Self { status: "error", data: None, error: Some(msg.into()) }
    }
}
```

Контракты сообщений Kafka — общие для продюсеров и консьюмеров (интеграционный «костяк» системы):

```

pub struct AnalysisRequest {
    pub request_id: Uuid,
    pub case_id:    Uuid,
    pub image_key:  String,          // ключ объекта в S3/MinIO
    pub source:     AnalysisSource,  // Doctor | Lab
    pub batch_id:   Option<Uuid>,
    pub patient_ethnicity: Option<String>,
    pub requested_at: DateTime<Utc>,
}

pub struct DiagnosisEntry {
    pub syndrome_id: Uuid,
    pub syndrome_name: String,
    pub confidence:  f64,    // [0.0, 1.0]
    pub rank:        i32,    // 1 = наиболее вероятный
}

pub struct AnalysisResult {
    pub request_id: Uuid,
    pub case_id:    Uuid,
    pub source:     AnalysisSource,
    pub batch_id:   Option<Uuid>,
    pub top_diagnoses: Vec<DiagnosisEntry>,
    pub processing_time_ms: u64,
    pub completed_at: DateTime<Utc>,
}

```

5.2. Карта сервисов

Сервис	Порт	Назначение	Хранилища	Kafka
auth	3001	Регистрация, вход, JWT, refresh, профиль, выход	PostgreSQL, Redis	—
case	3002	CRUD кейсов, presigned-загрузка, запуск анализа, сохранение диагнозов	PostgreSQL, MinIO	publish requests / consume results / publish notifications

labs	3003	Пакеты лабораторий, результаты, потребление, тарифы	PostgreSQL, Redis	publish requests / consume results
recognition	3004	«Инференс» (заглушка): top-10 синдромов	— (каталог в коде)	consume requests / publish results
notifier	3005	Список уведомлений, отметка прочитанным, рассылка e-mail	PostgreSQL	consume notifications
reference	3006	Справочник синдромов и НПО, поиск	PostgreSQL, Elasticsearch	—
billing	3007	Тарифы, подписки, учёт потребления, проверка квоты	PostgreSQL, Redis	—

5.3. Сервисы подробно

auth (3001)

Пароли хешируются **Argon2id**. При входе выдаётся пара токенов: короткоживущий access-JWT и refresh-токен (случайный UUID, в БД хранится его SHA-256-хеш). **POST /auth/refresh** выполняет **ротацию** (старый refresh удаляется, выдаётся новый). **DELETE /auth/logout** удаляет refresh из БД и помещает access в чёрный список Redis на остаток времени жизни. JWT подписывается RS256 (при наличии RSA-ключей) с откатом на HS256.

Хеширование пароля (Argon2id) и полезная нагрузка access-токена:

```
// регистрация: соль + Argon2id
let salt = SaltString::generate(&mut OsRng);
let password_hash = Argon2::default()
    .hash_password(body.password.as_bytes(), &salt)?
    .to_string();

// полезная нагрузка RS256 JWT
pub struct Claims {
    pub sub: String, // UUID пользователя
    pub email: String,
    pub role: String, // "doctor" | "lab" | "admin"
```

```
pub exp:  usize,    // срок действия (Unix-время)

pub iat:  usize,

}
```

case (3002)

Сердце врачебного сценария. Создаёт кейс и генерирует presigned PUT-URL (окно 15 минут) для прямой загрузки в MinIO. На подтверждении проверяет наличие объекта (HEAD), переводит кейс в `processing` и публикует запрос в Kafka. Фоновый consumer на `analysis.results` сохраняет диагнозы, ставит статус `completed` и публикует уведомление. Проверка владения: кейс доступен только создавшему его пользователю.

Подтверждение загрузки запускает асинхронный анализ (`confirm_upload`):

```
let case = load_owned_case(&state, auth.user_id, id).await?;

if case.status != "pending" {
    return Err(CaseError::InvalidState { current: case.status });
}

// объект действительно загружен в хранилище?
if !s3::check_object_exists(&state.s3, bucket, &case.image_key).await? {
    return Err(CaseError::ImageNotFound { key: case.image_key });
}

db::update_case_status(&state.pool, id, "processing").await?;

let request = AnalysisRequest {
    request_id: Uuid::new_v4(),
    case_id: id,
    image_key: case.image_key,
    source: AnalysisSource::Doctor,
    batch_id: None,
    patient_ethnicity: case.patient_ethnicity,
    requested_at: Utc::now(),
};

kafka::publish_analysis_request(&state.producer, request).await?; // → analysis.requests
```

labs (3003)

Принимает пакет ключей изображений, последовательно: проверяет роль `lab`, применяет **ограничитель частоты** (Redis, 10 запросов/мин на лабораторию), **проверяет квоту** через

биллинг (с 30-секундным кэшем в Redis), создаёт пакет и элементы, публикует по одному запросу на изображение. Эндпоинты потребления и тарифов проксируют биллинг-сервис.

billing (3007)

Хранит тарифы (засеяны Bronze/Silver/Gold/Platinum/Bundle), подписки и помесечный учёт. `check-quota` — внутренний эндпоинт: при исчерпании квоты возвращает 429 с заголовком `Retry-After` (секунды до начала следующего месяца). Счётчик потребления дублируется в Redis (быстрый путь) и в PostgreSQL (источник истины).

recognition (3004)

См. отдельный раздел 6. Кратко: каталог 50 синдромов, случайный top-10, имитация задержки 0,5–2 с, прямой эндпоинт `POST /recognize` для тестов + основной режим — consumer Kafka.

notifier (3005)

Consumer топика `notifications`: сохраняет уведомление в БД и пытается отправить e-mail через SMTP (в dev-режиме `localhost:1025` — только логирует намерение). HTTP-эндпоинты: список уведомлений пользователя и отметка прочитанным.

reference (3006)

Справочные данные только для чтения. Полнотекстовый поиск сначала пробует Elasticsearch, при недоступности или пустом индексе **прозрачно откатывается** на `ILIKE` в PostgreSQL. Карточка синдрома отдаётся вместе со связанными НРО-терминами.

5.4. Модель данных

Схема	Таблицы (ключевые поля)
auth	<code>users</code> (email, password_hash, role, name, organization), <code>refresh_tokens</code> (token_hash, expires_at)
cases	<code>cases</code> (user_id, patient_name/age/ethnicity, image_key, status), <code>diagnoses</code> (case_id, syndrome_name, confidence, rank)
labs	<code>batches</code> (lab_id, status, total/processed_items), <code>batch_items</code> (batch_id, image_key, case_id, status)
billing	<code>plans</code> (name, monthly_limit, overage_price_cents), <code>subscriptions</code> (lab_id, plan_id, status), <code>usage_records</code> (lab_id, month, count)

notifier `notifications` (user_id, type, title, body, read, metadata JSONB)

reference `syndromes` (name, omim_id, description, prevalence, inheritance), `hpo_terms` (hpo_id,
e name, definition, parent_id), `syndrome_hpo` (связка M:N)

5.5. API и шлюз

Все 24 операции описаны в рукописной спецификации **OpenAPI 3.1** (`docs/openapi.yaml`), к ней прилагается Swagger UI (в продакшн наружу не публикуется). Шлюз nginx маршрутизирует по префиксу: `/auth/` → auth, `/cases` → case, `/labs/` → labs, `/recognize` → recognition, `/notifications` → notifier, `/reference/` → reference, `/billing/` → billing; `/healthz` — liveness. Полная таблица операций — в Приложении А.

6. Ядро распознавания (ML): текущее состояние и цель

Текущее состояние — заглушка (stub). Сервис `recognition` не анализирует изображение. Алгоритм: перемешать каталог из 50 синдромов, взять первые 10, присвоить им убывающие «оценки уверенности» (rank-1 — случайно из диапазона [0,30; 0,95], каждый следующий — умножение на множитель из [0,50; 0,90], что гарантирует строгое убывание), вернуть результат. Перед ответом — имитация задержки инференса 0,5–2 с.

Ядро «инференса» MVP (`recognition/generator.rs`) — именно эту функцию заменит реальная модель:

```
pub fn generate_random_diagnoses(syndromes: &[SyndromeInfo]) -> Vec<DiagnosisEntry> {
    let mut rng = rand::rng();

    let mut pool: Vec<&SyndromeInfo> = syndromes.iter().collect();

    pool.shuffle(&mut rng); // перемешать каталог (изображение
НЕ анализируется)

    let mut confidence = rng.random_range(0.30..0.95); // оценка для ранга 1

    pool.iter().take(10).enumerate().map(|(i, s)| {
        if i > 0 {
            confidence *= rng.random_range(0.50..0.90); // каждый следующий ранг строго
ниже
        }

        DiagnosisEntry {
            syndrome_id: s.id,
            syndrome_name: s.name.to_string(),
            confidence,
            rank: (i + 1) as i32,
        }
    }).collect()
}
```

Это сознательное и честное проектное решение для MVP: оно позволяет полностью продемонстрировать **сквозной асинхронный конвейер** (загрузка → очередь → обработка → результат → уведомление → отображение), не дожидаясь готовности обученной модели. Каталог из 50 синдромов использует *стабильные UUID* (`Uuid::from_u128(1..=50)`), совпадающие с идентификаторами в справочной БД, поэтому результат корректно ссылается на карточки синдромов.

Почему замена модели не потребует переделки системы. Контракт сообщений уже промышленный: `AnalysisRequest` несёт ключ изображения, источник, опционально этничность пациента; `AnalysisResult` — список `DiagnosisEntry` (`syndrome_id`, `name`, `confidence` $\in [0,1]$, `rank`) и время обработки. Реальной модели достаточно подменить функцию генерации диагнозов внутри того же `consumer`-цикла — остальные сервисы, БД и фронтенд останутся без изменений.

Целевая модель (видение)

В целевой архитектуре Recognition работает на GPU-узлах, использует векторную БД эмбеддингов лица и реестр моделей (MLflow). По данным рыночного анализа, для конкурентоспособной модели потребуются: $\geq 17\,000$ изображений, покрытие 200+ синдромов, этническое разнообразие выборки, клиническая валидация (целевая точность `top-10` $\approx 90\%$) и публикации. Это — основной предмет следующего этапа разработки.

7. Фронтенд: Phenograph SPA

7.1. Технологический стек

Область	Технология
Каркас	React 19, Vite 8, TypeScript 6 (строгий режим)
Маршрутизация	React Router 7
Серверное состояние	TanStack Query (React Query) 5 — кэш, поллинг, инвалидация
Валидация форм	Zod 4 (без отдельной form-библиотеки)
Моки API	MSW 2 (Mock Service Worker) — демо без бэкенда
Иконки / стили	Lucide; чистый CSS с дизайн-токенами (без Tailwind/CSS-in-JS)

7.2. Архитектура клиента

- **API-клиент** (`lib/api`): обёртка `apiFetch` разворачивает конверт `ApiResponse`, подставляет Bearer-токен и при 401 выполняет **единый (single-flight) refresh** с повтором запроса; параллельные 401 разделяют один промис обновления.
- **Состояние данных**: хуки React Query на каждую предметную область; кейсы и пакеты в обработке **опрашиваются каждые 2 с**; уведомления — каждые 30 с.
- **Аутентификация** (`AuthContext`): токены в `localStorage`; на старте — попытка `/auth/me` с авто-обновлением; маршруты защищены охранниками (`ProtectedRoute` / `PublicOnlyRoute` / `RoleRoute`), навигация и домашний экран зависят от роли (врач → `/app/cases`, лаборатория → `/app/usage`).
- **Демо-режим**: при `VITE_USE MOCKS=true` MSW перехватывает запросы и обслуживает их из встроенной in-memory БД (кейсы авто-проходят `pending` → `processing` → `completed`). В продакшн-сборке моки выключены.

7.3. Дизайн-система

Визуальная идентичность задана набором CSS-токенов. Палитра — «природная», сдержанно-

клиническая:

Токен	Hex	Назначение
teal-bright	#068176	Основной акцент (кнопки, ссылки, фокус)
deep-teal	#114643	Заголовки, тёмные поверхности
midnight	#15282f	Основной текст, тёмные секции
moss / olive / forest	#a2b568 / #697431 / #475d2e	Природная зелёная гамма
paper	#faf8f2	Фон страницы (тёплый белый)

Типографика: **Fraunces** (заголовки, серифный «характер»), **Inter** (текст/интерфейс), **JetBrains Mono** (идентификаторы, технические значения). Сетка отступов с базой 4 рх, модульная типографическая шкала $\times 1,2$, аккуратные тени и анимации. Библиотека UI-примитивов: Button, Card, Form, StatusPill, ConfidenceBar, Spinner, EmptyState, Toast, Modal, Dropzone, Pagination и др.

Замечание о шрифтах на снимках. На живом стенде заголовок `CSP (style-src 'self')` блокирует загрузку шрифтов Google Fonts, поэтому на экранных снимках вместо Fraunces/Inter отображаются системные шрифты-заместители (серифный/без засечек). На внешний вид это влияет незначительно, но это — фактическое состояние развёртывания (см. раздел 10).

8. Безопасность

Что реализовано и проверено.

- **Аутентификация:** Argon2id для паролей; JWT RS256; ротация refresh-токенов; чёрный список access-токенов в Redis при выходе.
- **Авторизация (RBAC):** роли doctor/lab/admin в токене; проверка владения ресурсами (кейс/пакет — только владельцу).
- **Периметр:** TLS (Let's Encrypt) + редирект HTTP → HTTPS; CORS, ограниченный origin фронтенда; rate-limit на логин (nginx) и на пакеты лабораторий (Redis); заголовки CSP/HSTS; firewall ufw (открыты только 22/80/443); вся инфраструктура — только loopback.
- **SQL-инъекции:** все запросы используют параметризованные привязки `sqlx`.
- **XSS:** нет `dangerouslySetInnerHTML`; React экранирует вывод; строгая CSP как компенсация хранения JWT в `localStorage`.

Зафлагованные риски (требуют решения до широкого запуска).

- **Открытая регистрация:** любой может зарегистрироваться с любой ролью — нужна модерация/CAPTCHA/инвайты.
- **JWT в localStorage:** осознанный компромисс; митигируется строгой CSP, но уязвим к XSS.
- **MinIO buckets — public-read;** Kafka и Elasticsearch — одноузловые без аутентификации (допустимо только потому, что закрыты loopback на одной машине).
- **Смена пароля PostgreSQL** требует пустого тома данных (особенность инициализации).
- **Нет HA/реплик/бэкапов** — одна машина (см. раздел 9–10).

9. Развёртывание

Предусмотрено два контура развёртывания.

Продакшн: Kubernetes

Для промышленной эксплуатации **подготовлены и проверены манифесты Kubernetes** (см. §4.2, Рис. 15): сервисы — как Deployments за Ingress с TLS, stateful-компоненты (PostgreSQL, Kafka, Redis, MinIO, Elasticsearch) — как StatefulSets/тома в кластере. Это целевой способ развёртывания, обеспечивающий горизонтальное масштабирование и самовосстановление подов.

Демо-стенд: одна машина

Текущий публичный демо-стенд `rusted-apple.com` развёрнут упрощённым идемпотентным скриптом `deploy.sh` (10 шагов): генерация секретов и `.env` (сильные пароли БД/MinIO, RSA-ключи JWT, CORS на домен фронтенда) → подъём инфраструктуры в Docker → миграции → сборка `cargo build --release` и установка 7 systemd-юнитов (`Restart=always`, автозапуск) → засев демо-доступов → сборка и публикация фронтенда → конфигурация системного nginx (frontend/API/storage) → выпуск TLS-сертификатов (certbot) → настройка firewall (ufw). Все экранные снимки в отчёте сделаны именно с этого стенда.

Домен	Назначение
<code>frontend.rusted-apple.com</code>	SPA (статика из <code>/var/www/phenograph</code>)
<code>api.rusted-apple.com</code>	API (через шлюз → 7 сервисов)
<code>storage.*</code> (опц.)	MinIO для браузерных presigned-загрузок

Демо-доступы (засеяны в реальную БД через `/auth/register`):

Роль	E-mail	Пароль
Врач	<code>doctor@phenograph.test</code>	<code>demopass123</code>
Лаборатория	<code>lab@phenograph.test</code>	<code>demopass123</code>

10. Текущие ограничения и известные проблемы

Раздел приведён для полноты и честности картины. **Врачебный сценарий работает на живом стенде полностью** (вход → кейс → диагнозы), и именно он — «витрина» продукта. Ниже — ограничения, преимущественно в монетизационном (лабораторном) контуре и в части инфраструктуры.

Функциональные ограничения

- **Распознавание** — **заглушка** (раздел 6): текущая «уверенность» и состав диагнозов случайны.
- **Лабораторные диагнозы не сохраняются.** Пакет и его элементы доходят до статуса `completed`, но Labs-сервис не сохраняет сами диагнозы, а `case_id` элемента — это сгенерированная заглушка, не существующая в Case-сервисе. Поэтому диагнозы по элементам пакета недоступны для просмотра, а Case-сервис логирует «unknown case» на лабораторном трафике (оба сервиса слушают `analysis.results`).
- **Профиль** — **только чтение** (нет эндпоинта обновления).
- **Нет списка пакетов** на бэкенде — фронтенд хранит созданные id в `localStorage`.
- **Загрузка пакета принимает ключи, а не файлы**, и нет `presign`-эндпоинта для лабораторий — в продакшне объекты должны быть заранее размещены в хранилище.

Наблюдения на живом стенде

- **Страница биллинга падает** (ошибка `n.map is not a function`), `/labs/usage` отвечает `500`, а у демо-лаборатории нет активной подписки. Совокупно это **блокирует демонстрацию лабораторного/ монетизационного потока end-to-end** на текущем стенде.
- **CSP блокирует Google Fonts** (`style-src 'self'`) → сайт показывается системными шрифтами.

Инфраструктурные ограничения

- Одна машина: нет высокой доступности, репликации и резервного копирования.
- Kafka/Elasticsearch — одноузловые; MinIO — `public-read` (допустимо только за `firewall`).

11. Дорожная карта

1. **Реальная ML-модель распознавания** + векторная БД и реестр моделей; клиническая валидация и публикации.
2. **Завершение лабораторного контура:** сохранение диагнозов пакета, связка `case_id`, починка биллинг-страницы и `/labs/usage`, сценарий подписки.
3. **Локализация интерфейса на русский язык** — ключевое рыночное преимущество.
4. **Интеграция с МИС/ЭМК и экспорт в FHIR.**
5. **Production-grade инфраструктура:** манифесты Kubernetes уже подготовлены (§4.2/§9) — остаётся перевести на них живой стенд, добавить горизонтальное масштабирование, наблюдаемость (Prometheus/Grafana/Jaeger/ELK) и бэкапы.
6. **Контроль доступа:** закрытие открытой регистрации (инвайты/модерация), полноценная admin-роль.
7. **Перенос JWT в httpOnly-cookie** (снижение XSS-риска), усиление аутентификации инфраструктуры (Kafka/ES, приватные buckets).

12. Приложения

Приложение А. Полная карта API (24 операции)

Метод и путь	Сервис	Описание	Аутент.
POST /auth/register	auth	Регистрация пользователя	—
POST /auth/login	auth	Вход, выдача пары токенов	—
POST /auth/refresh	auth	Обновление токенов (ротация)	refresh
GET /auth/me	auth	Профиль текущего пользователя	JWT
DELETE /auth/logout	auth	Выход, отзыв токенов	JWT
POST /cases	case	Создать кейс + presigned-URL	JWT
GET /cases	case	Список кейсов (пагинация)	JWT
GET /cases/{id}	case	Кейс с диагнозами	JWT
PATCH /cases/{id}	case	Правка метаданных (только pending)	JWT
POST /cases/{id}/confirm-upload	case	Подтвердить загрузку, запустить анализ	JWT
POST /labs/batch	labs	Создать пакет анализа	JWT (lab)
GET /labs/results/{batch_id}	labs	Пакет и его элементы	JWT (lab)
GET /labs/usage	labs	Потребление за месяц	JWT (lab)
GET /labs/tiers	labs	Доступные тарифы	JWT
POST /recognize	recognition	Прямой инференс (заглушка, для тестов)	—

GET /notifications	notifier	Список уведомлений	JWT
PATCH /notifications/{id}/read	notifier	Отметить прочитанным	JWT
GET /reference/syndromes	reference	Список/поиск синдромов	JWT
GET /reference/syndromes/{id}	reference	Синдром + HPO-термины	JWT
GET /reference/hpo/search	reference	Поиск по HPO	JWT
GET /billing/plans	billing	Список тарифов	—
POST /billing/subscribe	billing	Оформить подписку	—*
GET /billing/usage/{lab_id}	billing	Потребление лаборатории	—*
POST /billing/check-quota	billing	Внутренняя проверка квоты	—*

* Биллинг-эндпоинты в текущем MVP не защищены отдельно (вызываются изнутри/проксируются); требует доработки.

Приложение В. Каталог синдромов распознавания (50)

Down, Turner, Noonan, Williams, Angelman, Prader-Willi, Cornelia de Lange, Kabuki, Rubinstein-Taybi, Treacher Collins, Marfan, Ehlers-Danlos, Fragile X, Rett, Apert, Crouzon, DiGeorge, Wolf-Hirschhorn, Cri du Chat, Patau, Edwards, Klinefelter, Sotos, Beckwith-Wiedemann, Russell-Silver, Stickler, Waardenburg, Coffin-Siris, Floating-Harbor, KBG, Mowat-Wilson, Phelan-McDermid, Costello, Cardiofaciocutaneous, CHARGE, Jacobsen, Smith-Magenis, Potocki-Lupski, Kleefstra, Koolen-de Vries, Schinzel-Giedion, Marshall-Smith, Weaver, Pallister-Killian, Bohring-Opitz, Wiedemann-Steiner, Nicolaides-Baraitser, Genitopatellar, DOORS, Bainbridge-Ropers.

Приложение С. Доступы и ссылки

- Фронтенд: <https://frontend.rusted-apple.com>
- API: <https://api.rusted-apple.com> (liveness: </healthz>)
- Демо-врач: doctor@phenograph.test / [demopass123](#)
- Демо-лаборатория: lab@phenograph.test / [demopass123](#)

Приложение D. Структура репозитория (ключевое)

- [crates/oblivion-common/](#) — общие контракты (конверт, ошибки, типы, Kafka).
- [services/{auth,case,labs,recognition,notifier,reference,billing}/](#)

микросервисов.

- `migrations/` — SQL-миграции по схемам.
 - `gateway/nginx.conf` — конфигурация API-шлюза.
 - `frontend/app/` — продакшн-SPA (React/Vite/TS).
 - `docs/openapi.yaml` — спецификация API; `docs/DEPLOY.md` — инструкция развёртывания.
 - `design/` — проектные материалы: целевая архитектура и рыночный анализ.
-

Phenograph / Oblivion — отчёт о продукте (MVP). Подготовлено на основе исходного кода репозитория, проектных материалов `design/` и наблюдений на живом стенде. Не является медицинским изделием — инструмент поддержки клинических решений.